DAY 11

Object-Oriented Programming II

INTERMEDIATE PHASE • SESSION 11 OF 30

2 hrs	4	3	22
Duration	Topics	Concept files	Practice problems

Inheritance & super() · Polymorphism & Duck Typing · Encapsulation & @property · Dunder (Magic) Methods

1. Session Overview

Goal: Extend Day 10's introduction to classes by teaching students how classes relate to one another (inheritance), how the same method name can behave differently across classes (polymorphism), how to protect internal object state (encapsulation), and how to make custom objects behave like built-in Python types (dunder methods).

Outcome: By the end of this session, students will be able to build a small class hierarchy with `super()`, write polymorphic functions that work across related and unrelated classes, protect sensitive attributes with naming conventions and `@property`, and implement `__str__`, `__repr__`, `__len__`, and `__eq__` on their own classes.

Timeline (2 hours)

Time	Segment	Focus
0:00 - 0:05	Recap & Hook	Quick Day 10 review; pain point: duplicated class code
0:05 - 0:35	Step 1	Inheritance & super() — Employee -> Manager -> SeniorManager
0:35 - 1:00	Step 2	Polymorphism & duck typing — PaymentMethod subclasses
1:00 - 1:10	Break	5-10 min stretch break
1:10 - 1:35	Step 3	Encapsulation — protected/private attributes, @property
1:35 - 1:55	Step 4	Dunder methods — __str__, __repr__, __len__, __eq__
1:55 - 2:00	Wrap-up	Recap, mini-project check-in, Day 12 preview

TIP

Instructor note: Day 11 is conceptually denser than Day 10. If running short on time, the dunder methods section (Step 4) can be trimmed to just __str__ and __eq__, with __repr__ and __len__ left as self-study via the practice problems.

2. Prerequisites & Preparation

Requirements

Requirement	Details
Completed	Day 10 (OOP I: classes, __init__, self, instance variables/methods)
Python version	3.12 (as installed on Day 1)
Editor	VS Code with Python + Pylance extensions
Files needed	day11_01_inheritance_super.py, day11_02_polymorphism_encapsulation.py, day11_03_dunder_methods.py

Instructor Pre-Session Checklist

#	Checklist Item
1	Re-read Day 10 notebook to bridge terminology (class, object, self, __init__)
2	Open all three Day 11 .py files in VS Code, test run each beforehand
3	Prepare the whiteboard/slide for the Employee -> Manager -> SeniorManager diagram
4	Have the AttributeError demo ready (name-mangling) — it surprises students every time
5	Queue up the practice problem files for the second half of class

3. Step-by-Step Instruction Guide

STEP 1 — Inheritance & super() (30 min)

What to Say

"Yesterday every class we wrote stood alone. But in the real world, an Employee and a Manager share a lot — name, ID, salary. A Manager IS an Employee, plus extra responsibilities. Let's stop copy-pasting and let Python express that relationship directly with inheritance."

```
class Employee:
    def __init__(self, name, employee_id, monthly_salary):
        self.name = name
        self.employee_id = employee_id
        self.monthly_salary = monthly_salary

    def describe(self):
        return f"{self.name} earns Rs.{self.monthly_salary}/month"


class Manager(Employee):
    def __init__(self, name, employee_id, monthly_salary, team_size):
        super().__init__(name, employee_id, monthly_salary)
        self.team_size = team_size

    def describe(self):
        base = super().describe()
        return f"{base}, leading a team of {self.team_size}"
```

```
priya = Manager("Priya Sharma", "MGR201", 90000, team_size=8)
print(priya.describe())
# Output: Priya Sharma earns Rs.90000/month, leading a team of 8
```

"Notice two things: Manager(Employee) means Manager inherits everything from Employee. And super().__init__(...) lets us reuse the parent's setup code instead of retyping it."

Common Errors

Error	Cause	Fix
TypeError: missing required positional argument	Forgot to pass all args to super().__init__()	Match the parent's __init__ signature exactly
AttributeError on inherited method	Misspelled the parent class name in class Child(Parent)	Double-check the parent class name and spelling

Error	Cause	Fix
Infinite recursion / RecursionError	Calling self.describe() instead of super().describe() inside an override	Use super() to call the parent's version, not self

DONE WHEN

Done when: students can create a child class, call super().__init__() correctly, and override a method while still reusing the parent's version with super().

STEP 2 — Polymorphism & Duck Typing (25 min)

What to Say

"What if we have totally different payment methods — UPI, Card, Cash — and we want to process payments without writing a separate if/elif chain for each type? If every class defines the SAME method name, Python will call the right one automatically. That's polymorphism."

```
class UPIPayment:
    def pay(self, amount):
        return f"Paid Rs.{amount} via UPI"

class CardPayment:
    def pay(self, amount):
        return f"Paid Rs.{amount} via Credit Card"

methods = [UPIPayment(), CardPayment()]
for method in methods:
    print(method.pay(500))
# Output:
# Paid Rs.500 via UPI
# Paid Rs.500 via Credit Card
```

"Notice UPIPayment and CardPayment don't even share a parent class here — Python doesn't care, as long as the object HAS a .pay() method. We call this duck typing: if it walks like a duck and quacks like a duck..."

Common Errors

Error	Cause	Fix
AttributeError: object has no attribute 'pay'	An object in the loop is missing the expected method	Ensure every class in the polymorphic list defines that method
Wrong output / same behaviour everywhere	Forgot to override the method in the child class	Re-define the method inside the child class body

DONE WHEN

Done when: students can write a loop that calls the same method name on a list of different objects and correctly predict each output.

STEP 3 — Encapsulation & @property (25 min)

What to Say

"Sometimes we don't want outside code messing directly with an object's internal state — imagine someone setting a bank balance to a negative number! Python gives us naming conventions: a single underscore _x means 'internal use, please don't touch', and a double underscore __x triggers name mangling, making accidental access much harder."

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.__balance = balance   # name-mangled to _BankAccount__balance

    def deposit(self, amount):
        self.__balance += amount
        return self.__balance

account = BankAccount("Arjun", 5000)
account.deposit(1500)
print(account.__balance)
# Output: AttributeError: 'BankAccount' object has no attribute '__balance'
```

"This surprises everyone the first time! Let it land, then show @property — it lets us expose a controlled 'door' into that private data."

```
class Student:
    def __init__(self, name, marks):
        self._marks = marks

    @property
    def marks(self):
        return self._marks

    @marks.setter
    def marks(self, value):
        if not (0 <= value <= 100):
            raise ValueError("Marks must be between 0 and 100")
        self._marks = value

priya = Student("Priya", 82)
priya.marks = 95     # runs through the validation in the setter
print(priya.marks)
# Output: 95
```

Common Errors

Error	Cause	Fix
AttributeError on __x access from outside	Expected behaviour due to name mangling	Use a public method or @property instead of touching __x directly
AttributeError: can't set attribute	Tried to assign to a @property with no setter defined	Add an @x.setter method, or use a different internal attribute
RecursionError in @property getter/setter	Setter named the same as the stored attribute (self.marks = value inside the marks setter)	Store the real value under a different name, e.g. self._marks

WATCH OUT

Watch out: students often write the property setter so it assigns back to the same property name (self.marks = value inside def marks(self, value)), causing infinite recursion. Walk through this on the board before they hit it independently.

DONE WHEN

Done when: students can explain the difference between _x and __x, and can write a class with a validated @property setter.

STEP 4 — Dunder (Magic) Methods (20 min)

What to Say

"Have you noticed print(my_object) shows something ugly like <__main__.Task object at 0x...>? We can fix that. Dunder methods — double underscore methods — are hooks Python calls automatically. Define __str__ and print() will use it."

```
class Task:
    def __init__(self, title, priority):
        self.title = title
        self.priority = priority

    def __str__(self):
        return f"Task('{self.title}', priority={self.priority})"

    def __eq__(self, other):
        return self.title == other.title and self.priority == other.priority

task1 = Task("Submit report", "High")
print(task1)
# Output: Task('Submit report', priority=High)

task2 = Task("Submit report", "High")
print(task1 == task2)
# Output: True (compares VALUES, not memory identity)
```

"Without __eq__, task1 == task2 would be False even with identical data, because Python compares memory addresses by default. __eq__ lets us redefine what 'equal' means for our own class."

Common Errors

Error	Cause	Fix
print() still shows memory address	__str__ not defined, or has a typo (e.g. __strr__)	Check spelling: exactly __str__ with double underscores both sides
TypeError in __len__	__len__ doesn't return an int	Make sure __len__ returns a plain integer
== always returns False	__eq__ compares self to other incorrectly, or other lacks the same attributes	Check attribute names match and isinstance(other, ClassName) if needed

DONE WHEN

Done when: students can implement __str__ and __eq__ on a custom class and explain the difference between __str__ (user-facing) and __repr__ (developer-facing).

4. Session Wrap-Up

Student Checklist

#	Can you...
1	Create a child class that inherits from a parent and calls super().__init__()?
2	Override a method in a child class while still reusing the parent's version?
3	Write a function that works polymorphically across multiple classes?
4	Explain the difference between _protected and __private (name-mangled) attributes?
5	Write a class with a validated @property setter?
6	Implement __str__, __repr__, __len__, and __eq__ on a custom class?

Key Concepts Recap

Inheritance: class Child(Parent) reuses and extends behaviour; super() calls the parent's version of a method.
Polymorphism: the same method name behaves differently per class; duck typing means Python only cares whether the method exists, not the class hierarchy.
Encapsulation: _x signals 'internal use' by convention; __x triggers name mangling; @property exposes controlled, validated access to internal state.
Dunder methods: __str__ (user-facing), __repr__ (developer-facing), __len__ (supports len()), __eq__ (value-based equality) let custom classes behave like built-in types.

Next Day Preview

Day 12 — Modules & Packages: import / from...import, installing packages with pip and uv add, the if __name__ == "__main__": guard, and key standard library modules (os, sys, math, datetime, pathlib).

Troubleshooting Notes

WATCH OUT

If a student's super().__init__() call raises a TypeError about argument count, have them print the parent class's __init__ signature first (or check it in the IDE) before adjusting the call.

5. Quick Reference Card

Daily Workflow Commands

```
$ cd python-course/day11
$ uv venv                    # if not already active
$ source .venv/bin/activate  # macOS/Linux
$ .venv\Scripts\activate     # Windows
$ python day11_01_inheritance_super.py
$ python day11_02_polymorphism_encapsulation.py
$ python day11_03_dunder_methods.py
```

VS Code Shortcuts

Shortcut	Action
Ctrl/Cmd + `	Open integrated terminal
F5	Run current file with debugger
Ctrl/Cmd + S	Save file
Shift + Alt + F	Format document (with Ruff)
Ctrl/Cmd + /	Toggle line comment

Tool / Concept Summary

Concept	Syntax	Use Case
Inheritance	class Child(Parent):	Share/extend behaviour from a parent class
super()	super().__init__(...)	Reuse parent constructor/methods
Polymorphism	same method name per class	One interface, many behaviours
Protected attr.	self._x	Signals internal use (convention only)
Private attr.	self.__x	Name-mangled, harder to access externally
Property	@property / @x.setter	Controlled, validated attribute access
__str__	def __str__(self):	Friendly text for print()/str()
__repr__	def __repr__(self):	Unambiguous developer-facing text
__len__	def __len__(self):	Enables len(obj)
__eq__	def __eq__(self, other):	Enables value-based == comparison